

Задача В. Строка из палиндромов

Подсчитаем количество вхождений каждой буквы в строку. Каждая буква, которая встречается нечетное количество раз, должна быть серединой некоторого палиндрома нечетной длины (возможно, длины 1), так как для нее нет парной буквы.

Тогда минимальное количество палиндромов, которое можно составить, равно количеству букв, встречающихся нечетное количество раз. Обозначим их количество за k . Если это число больше m , то ответ – NO.

Иначе, если $k \leq m$, то построим k палиндромов, в центрах которых – нечетные буквы. Если $k < m$, то оставшиеся $m - k$ палиндромов построим, используя оставшиеся буквы, количество которых четно.

Так как $m \leq n$, то еще $m - k$ букв в слове точно есть. Получить ровно m палиндромов можно, например, следующим образом:

- Если $m - k$ нечетно, то сделаем $m - k - 1$ (четное число) букв серединами 1-буквенных палиндромов. Количество оставшихся букв четно, из них можно получить еще 1 палиндром.
- Иначе сделаем $m - k - 2$ (также четное число) 1-буквенных палиндромов, а оставшиеся буквы распределим в 2 палиндрома поровну.

Ответ в этих случаях будет YES.

Задача С. Судьба

Для решения можно воспользоваться структурой данных `std::set`. Посчитаем степени вершин и сложим в сет пары $(\deg(v), v)$. На каждом шаге будет доставать все вершины максимальной степени и уменьшать степени их соседей на один. Доставать вершины можно с помощью итератора `rbegin`, уменьшать степень можно явно удаляя вершину из сета и добавляя вершину со степенью, уменьшенной на один. Для этого полезно хранить текущую степень каждой вершины в массиве. Количество выполненных итераций будет ответом.

Каждая вершина будет уничтожена ровно один раз и в этот момент будут рассмотрены все её соседи. Количество вершин n , количество рёбер m , стоимость обращения к `std::set` $O(\log n)$. Асимптотика решения: $O((n + m) \log n)$.

Задача F. Метро в Берляндии

Заметим, что после каждого запроса схема метро остается деревом. Тогда время, за которое можно пройти все метро и вернуться на начальную станцию составляет $2 \cdot$ (количество ребер в дереве) минут. Но нам не нужно возвращаться на начальную станцию. В таком случае, если зафиксировать начальную и конечную станцию, то время за которое можно посетить все станции хотя-бы один раз составляет $2 \cdot$ (количество ребер в дереве) - (расстояние от начальной до конечной станции). Чтобы минимизировать эту величину, нужно выбрать начальную и конечную станцию так, чтобы расстояние между ними было максимальное. Такую величину еще называют диаметром дерева. Задача свелась к тому, что нам нужно уметь поддерживать количество ребер и диаметр в дереве после каждого запроса.

Научимся решать задачу, если у нас запросы только первого типа. Будем поддерживать концы диаметра дерева (пусть это будут вершина a и b). Представим, что мы добавили вершину c . Тогда нашим новым диаметром станет либо (a, b) , либо (a, c) , либо (b, c) . Это следует из того, что если подвесить дерево за предок вершины c , то самой глубокой вершиной будет вершина a или b (Доказательство этого факта остается за читателями). И тогда из алгоритма поиска диаметра дерева через два BFS, получаем вышеупомянутый результат.

Теперь мы заметим, что каждая станция имеет отрезок, на котором это станция существует. Заведем дерево отрезков, а в нодах будем хранить список станций (если нода отвечает за отрезок $[l, r]$, то все станции обязаны существовать на этом отрезке). Сделаем обход по дереву обходу отрезков. Будем поддерживать количество ребер и текущий диаметр дерева. Перед тем, как зайти в ноду скопируем значения количества ребер и диаметра дерева из предка. Затем обновим эти параметры с учетом списка вершин. Как только мы окажемся в i листе дерева отрезков, мы получим ответ на i запрос.